



Realtime, Permissions & Routing

Lecture 8: WebSockets, runtime permissions, navigation

Who's teaching this course

Dinu-Ștefan Rusu

dinu_stefan.rusu@upb.ro

Microsoft Teams course channel

rusudinu.com

45+

Flutter apps shipped

400k

installs across them

Appeared in, spoken at & partnered with



TODAY

What you should leave with



Realtime transport

Which of WebSocket, SSE, long-polling and MQTT fits a problem, and the handshake that makes a socket



Connections that survive

Backoff, heartbeats, wss://, and what the OS does to your socket when the app leaves the screen



Permissions, end to end

Manifest and plist entries, Android 13+ granular media, and every state permission_handler returns



Routing with go_router

Routes as data, parameters, nested shells, plus a redirect guard wired to your auth state

PART 1 · REALTIME

Keeping a connection open

WebSockets, and the three other ways to push data to a phone

Why HTTP alone cannot push

HTTP is request–response. The client asks, the server answers, the exchange is over. Keep-alive reuses the TCP connection, but the server still has no way to speak first.

So apps fake it by polling: ask every two seconds, and be told “nothing new” ninety-nine times out of a hundred. You pay for a full request (headers, TLS bookkeeping, a server handler) every time, on every device.

	Polling over HTTP	WebSocket
Connection	a request per poll, or a reused keep-alive socket	one socket, opened once, held open
Who may speak	the client only	either side, at any moment
Worst-case latency	one full poll interval	one network hop
Cost while idle	a request every interval, forever	a few bytes of keepalive

A WebSocket is not “faster HTTP”. It is a different conversation shape. Use it when the server has data before the client would have asked for it.

The handshake: HTTP 101 Switching Protocols

```
GET /chat HTTP/1.1
Host: realtime.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13

HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=

// same TCP connection, now carrying WebSocket frames
```

What just happened

A WebSocket starts life as an ordinary HTTP request. That is why it travels over port 443 and through every proxy and firewall that already allows HTTPS.

The 101 response ends the HTTP part. Nothing is re-negotiated; the same TCP connection is handed over to a binary framing protocol (RFC 6455).

Because the opening request is HTTP, it carries cookies, an Origin header and a query string, which is how you authenticate the socket.

Four ways to push, and when each one fits

	Direction	Runs over	Reach for it when
WebSocket	full duplex	one TCP socket, HTTP upgrade	both sides talk: chat, presence, collaborative editing, games
SSE	server → client	a plain HTTP response that never ends	a one-way feed. Auto-reconnect and event ids are in the browser spec, free
Long polling	server → client	ordinary HTTP requests	a restrictive network blocks everything else. The universal fallback
MQTT	pub/sub, both ways	TCP, or tunneled over WebSocket	devices: tiny frames, QoS levels, retained messages, last-will messages

MQTT is the one you will meet on hardware. A broker fans one sensor reading out to every subscriber, and a 2-byte header suits a device with 520 KB of RAM. → Lecture 11: a TinyGo program on an ESP32-C3 publishes readings, and your Flutter app subscribes with `mqtt_client`.

web_socket_channel: the whole lifecycle

```
// 1. connect. wss:// only. connect() itself is lazy.
final channel = WebSocketChannel.connect(
  Uri.parse('wss://realtime.example.com/chat'),
);
await channel.ready; // throws if the handshake fails
// 2. listen. One stream, one subscription.
final sub = channel.stream.listen(
  (data) => onMessage(data as String),
  onError: (e, st) => scheduleReconnect(),
  onDone: () => scheduleReconnect(),
);
// 3. send. String or List<int>, nothing else.
channel.sink.add(jsonEncode({'type': 'msg', 'body': text}));
// 4. close. In dispose(), every time.
await sub.cancel();
await channel.sink.close(status.goingAway);
```

The two lines everyone forgets

`await channel.ready`

`connect()` returns instantly and never throws. Without `ready` you find out the server rejected you three screens later, as a stream error.

`sink.close()` in `dispose()`

A socket is not garbage-collected away. Leave it open and every abandoned screen leaks a radio wake-up, a server connection slot and battery.

`channel.closeCode` tells you why it ended: 1000 clean, 1006 abnormal (no close frame, the usual mobile case).

wss://, and who is allowed to connect

ws:// is plaintext.

Every frame (messages, ids, the token in your query string) crosses shared Wi-Fi in the clear, and any middlebox can rewrite it. wss:// is the same protocol inside TLS, on port 443.

You do not get a choice anyway: iOS App Transport Security and Android's cleartext policy both block ws:// by default. Do not add a platform exception to work around it.

A socket is authenticated once, at the handshake, and then trusted for hours.

So the server must check the token on the upgrade request, and must re-check it when it expires rather than trusting the connection forever.

Short-lived tokens, re-checked. A socket opened at nine with a one-hour token must not still be trusted at noon. The server closes it and the client reconnects with a fresh one.

Three ways to carry the token

Query string

?token=... is simplest and works in browsers, but it lands in server access logs. Use short-lived tokens.

Subprotocol header

Sec-WebSocket-Protocol carries the credential. Not logged; awkward in some proxies.

First message

Connect anonymously, send an auth frame, and let the server close the socket if it does not arrive within a second.

A dead socket still looks open

TCP does not tell you the peer is gone. It tells you when the peer sends a FIN or an RST, and a phone that has moved out of Wi-Fi range sends neither.

Carrier NAT rebinds your port after a couple of minutes of silence. A Wi-Fi → LTE handover changes your source address mid-connection. The radio sleeps. Every one of those leaves a socket that is open on your side and gone on the server's.

OS TCP keepalive defaults to two hours, which is far too long to detect this.

So you prove liveness yourself: send a ping, expect a reply, and treat silence as a closed connection.

A half-open connection is worse than a closed one. Messages you “sent” sit in a kernel buffer and the UI shows them as delivered.

```
// dart:io only: real protocol ping/pong frames
IOWebSocketChannel.connect(uri,
  pingInterval: const Duration(seconds: 20));

// Anywhere, incl. web: your own heartbeat
const beat = Duration(seconds: 20);
_hb = Timer.periodic(beat, (_) {
  if (_awaitingPong) {
    reconnect(); // no pong: treat it as closed
    return;
  }
  _awaitingPong = true;
  channel.sink.add({'t':"ping"});
});
```

Reconnecting without taking down your server

```
// Never reconnect in a tight loop: it overloads your own server.
Duration _backoff() {
    const base = Duration(seconds: 1), cap = Duration(minutes: 2);
    final expo = base * math.pow(2, _attempt); // 1s 2s 4s 8s ...
    final ceiling = expo > cap ? cap : expo;
    // Full jitter: pick anywhere in [0, ceiling].
    return Duration(milliseconds: _rng.nextInt(ceiling.inMilliseconds));
}

void scheduleReconnect() {
    if (_disposed || (_timer?.isActive ?? false)) return;
    _attempt++;
    _timer = Timer(_backoff(), _connect);
}

void onReady() => _attempt = 0; // reset once a connection sticks
```

Why jitter, not just doubling

A server restarts. Ten thousand clients notice at the same instant, so they all retry at $t+1s$, then all at $t+2s$, and overload it again. That is a thundering herd, and plain exponential backoff does not fix it.

Full jitter spreads the same clients evenly across the whole window.

Reset the counter on a connection that lasted, not on one that merely opened, because a server that accepts and immediately drops the connection would otherwise reset it every time.

When the app goes to the background

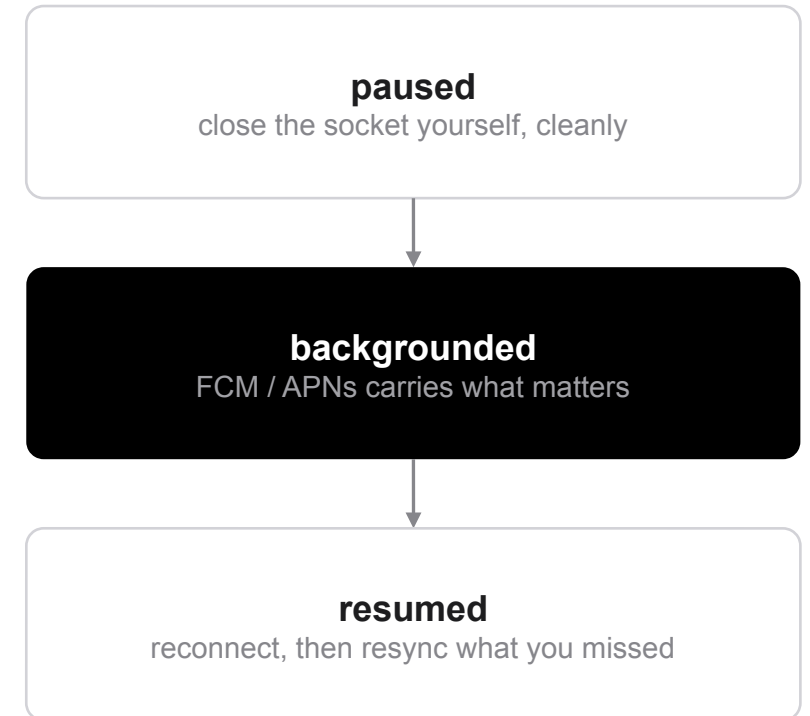
iOS gives a backgrounded app a few seconds and then suspends the process. Your isolate stops; your socket stops being serviced. No close frame is sent, so the server simply stops receiving from you.

Android does the same more gradually, through Doze and background restrictions.

A WebSocket is a foreground affordance.

Anything that must reach the user while the app is away is a push notification, FCM or APNs, not a frame on your stream. The socket only serves the live screen.

This is a common realtime bug on mobile. It works while the app is in the foreground and breaks as soon as the phone is locked. Handle it in `AppLifecycleListener`, and give every message a server-side id so resync is a gap query, not a guess.



PART 2 · PERMISSIONS

Asking the OS for the device

Manifest, plist, and every answer `permission_handler` can give you

Two steps, and both are mandatory

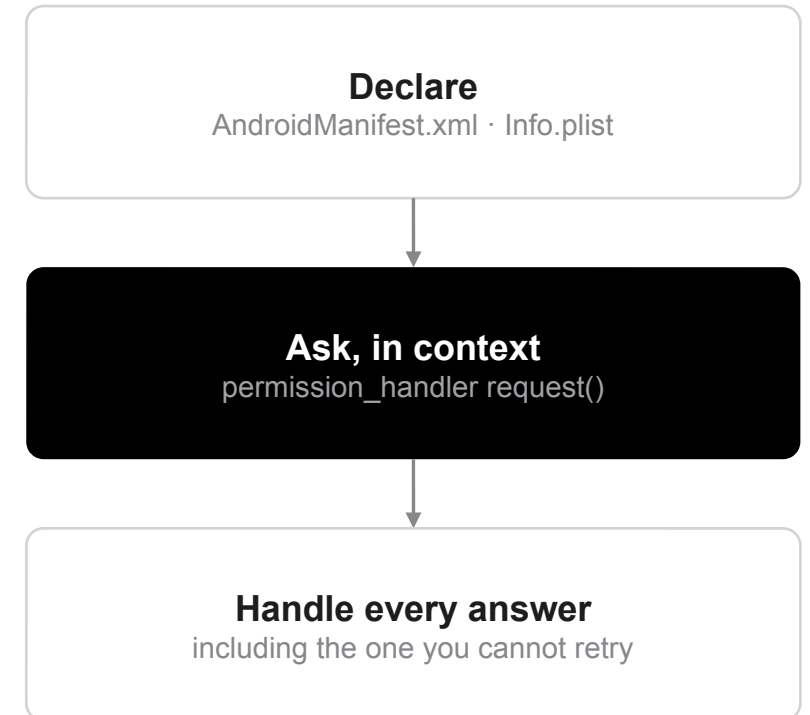
Step one is static. You declare, at build time, what your app might ever ask for. Android reads it from the manifest; iOS reads it from Info.plist, together with the exact sentence the user will be shown.

Step two is at runtime. The user is asked, once, in a system dialog you cannot style, move or re-word, and can say no.

Skip step one and step two does not fail gracefully.

On Android an undeclared permission is simply denied, forever, with no dialog. On iOS a missing usage-description string is not a warning: the app is terminated the moment it touches the API, and App Store review rejects the build.

Permissions are a build-config problem before they are a code problem.



Android: AndroidManifest.xml

```
<!-- android/app/src/main/AndroidManifest.xml -->
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.CAMERA"/>
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>

    <uses-permission android:name="android.permission.READ_MEDIA_IMAGES"/>
    <uses-permission android:name="android.permission.READ_MEDIA_VIDEO"/>
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
        android:maxSdkVersion="32"/>

    <uses-feature android:name="android.hardware.camera"
        android:required="false"/>
```

Line by line

Declaring is not requesting. These entries only make the runtime request legal.

`maxSdkVersion="32"` keeps the old storage permission for old phones without asking for it on Android 13+, where it no longer exists.

`uses-feature with required="false"` keeps the app installable on hardware without a camera. Leave it out and Play filters those devices away.

Flutter merges the plugin manifests into yours at build time, so check the merged manifest in build/ before you ship.

iOS: Info.plist

```
<!-- ios/Runner/Info.plist -->
<key>NSCameraUsageDescription</key>
<string>MapChat uses the camera to send photos in a chat.</string>

<key>NSMicrophoneUsageDescription</key>
<string>MapChat records voice notes while you hold the button.</string>

<key>NSLocationWhenInUseUsageDescription</key>
<string>MapChat shows friends near you while the app is open.</string>

<key>NSLocationAlwaysAndWhenInUseUsageDescription</key>
<string>MapChat alerts you when a friend arrives nearby.</string>

<key>NSPhotoLibraryUsageDescription</key>
<string>Pick a photo you already have and send it.</string>
```

The string is the product

That sentence is printed inside the system dialog. It is the only explanation you get, and you get it once.

Say what the user gains, not what the app needs. “To access your camera” earns a no; the sentences above earn a yes.

A missing key causes a hard crash: not an exception you can catch, but a termination.

`permission_handler` also needs the matching `PERMISSION_` macros enabled in `ios/Podfile`, or every permission you did not opt into reports `permanentlyDenied`.

Android 13+ split storage into media types

One coarse “read everything” permission became several narrow ones, and notifications stopped being granted at install.

	Before: API 32 and below	Android 13+ (API 33+)
Photos	READ_EXTERNAL_STORAGE	READ_MEDIA_IMAGES
Video	READ_EXTERNAL_STORAGE	READ_MEDIA_VIDEO
Audio and music	READ_EXTERNAL_STORAGE	READ_MEDIA_AUDIO
Notifications	granted at install, silently	POST_NOTIFICATIONS, a runtime prompt
Part of the library	not possible	READ_MEDIA_VISUAL_USER_SELECTED, API 34+

POST_NOTIFICATIONS is the one that matters for this course. Your FCM push, which is what reaches the user while the socket is closed, silently delivers nothing on an Android 13+ phone until the user grants it at runtime.

Every answer you have to handle

	What it means	What you do about it
isGranted	you may call the API	proceed
isDenied	asked and refused; on Android you may ask again	show a rationale, then request() once more
isPermanentlyDenied	Android: “Don’t ask again”. iOS: any refusal, ever.	request() is now a no-op; use openAppSettings()
isRestricted	blocked by policy: Screen Time, MDM, parental controls	hide the feature; there is no path forward
isLimited	a subset the user picked: iOS Photos, Android 14+ media	treat as granted, and offer “select more photos”
isProvisional	iOS notifications: delivered quietly, no prompt shown	you are already delivering; ask for the upgrade later

isLimited and isProvisional are not edge cases. They are the default outcome of two common iOS prompts, and a `status.isGranted` check treats both as a refusal.

permission_handler, including the dead end

```
Future<bool> ensureCamera(BuildContext context) async {  
  var status = await Permission.camera.status;  
  if (status.isGranted || status.isLimited) return true;  
  
  // iOS prompts once only. Show your rationale before requesting.  
  if (status.isDenied) {  
    if (!await showRationaleSheet(context)) return false;  
    status = await Permission.camera.request();  
  }  
  
  if (status.isPermanentlyDenied || status.isRestricted) {  
    // request() does nothing now. Settings is the only route.  
    if (await confirmOpenSettings(context)) await openAppSettings();  
    return false;  
  }  
  return status.isGranted || status.isLimited;  
}
```

Three traps

On Android, `permanentlyDenied` is only meaningful after you have asked at least once. Before that you get denied, so a check-on-launch reports the wrong thing.

`openAppSettings()` hands control away completely. The user may come back granted, denied, or not at all, so re-read the status on resume.

There is no API to open the system prompt twice, so a skipped rationale can cost you the permission permanently.

How to get a permission granted



Ask at the moment of need

Never on launch. Request the camera when the user taps the camera button, where the reason is obvious



Show your own screen first

A dismissible in-app sheet before the system dialog. If they say no to yours, you have spent nothing



Degrade, don't dead-end

Camera denied? Offer the gallery. Location denied? Let them type a city name. The feature still has to work



Give them a way back

When permanentlyDenied, show a button that opens Settings, not an error message with no next step



Ask for the narrower one

ACCESS_COARSE_LOCATION and READ_MEDIA_IMAGES are accepted more often than the broader permissions



Never ask for what you don't use

Store reviewers check declared permissions against what the app actually does. An unused entry is a rejection risk

PART 3 · ROUTING

Navigation as data

go_router: routes, parameters, shells, and a guard wired to auth

Navigator 1.0, and why the docs steer you off it

```
// Imperative. Still perfectly fine for a local dialog or sheet.  
Navigator.of(context).push(  
  MaterialPageRoute(builder: (_) => ChatScreen(id: id)),  
);  
  
// Named routes: no longer recommended by the Flutter team.  
MaterialApp(routes: {'/chat': (_) => const ChatScreen()});  
Navigator.pushNamed(context, '/chat');
```

What it cannot do

Deep links. A notification that should open `/chat/42` has to be turned into a sequence of pushes, by hand, in the right order.

Whole-stack changes. “Sign the user out and throw away every screen behind them” is awkward at best.

The web. Named routes do not become URLs, so the address bar and the browser back button are wrong.

Navigator 2.0 fixed all three, but its API was hard to use directly. `go_router` is the Flutter team's answer: it implements Navigator 2.0 for you and gives you a URL-shaped API on top.

go_router: your navigation, as data

```
final router = GoRouter(  
  initialLocation: '/',  
  routes: [  
    GoRoute(path: '/', builder: (c, s) => const HomeScreen()),  
    GoRoute(path: '/login', builder: (c, s) => const LoginScreen()),  
    GoRoute(  
      path: '/chat/:id',  
      builder: (c, s) => ChatScreen(  
        roomId: s.pathParameters['id']!,  
        draft: s.uri.queryParameters['draft'],  
      ),  
    ),  
  ],  
  errorBuilder: (c, s) => NotFoundScreen(uri: s.uri),  
);
```

Stack = f(state)

You stop saying “go here” and start declaring what exists. The router turns a location into a stack.

```
MaterialApp.router(routerConfig: router)
```

One line wires it in, and the same tree then works from a deep link, a push notification and a browser URL bar.

`context.go()` replaces the stack. `context.push()` adds to it. Use `go` for destinations and `push` for detours.

Three ways to carry data into a route

	Path	Navigate with	Read it back as	Survives a link?
Path param	/chat/:id	context.go('/chat/42')	state.pathParameters['id']	yes
Query param	/search	context.go('/search?q=ws')	state.uri.queryParameters['q']	yes
Extra	any	context.go('/chat/42', extra: room)	state.extra as Room	no

Pass ids, not objects. extra is an in-memory pointer: it does not survive a cold start from a notification, a browser refresh, or Android killing your process. Put the id in the URL and look the object up from your state manager on the other side, so the screen works however the user arrived. (→ Lecture 4, state management)

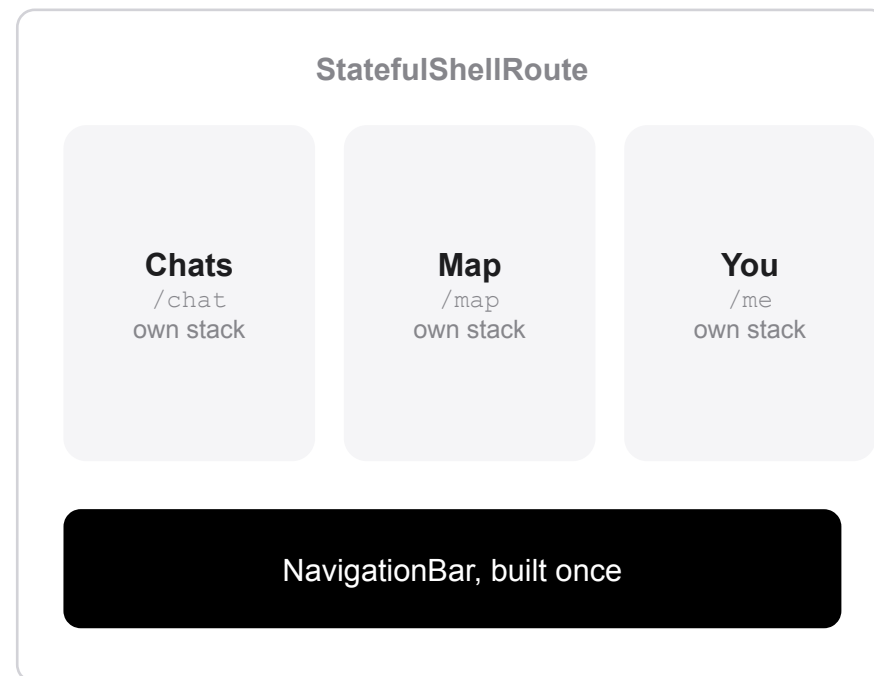
ShellRoute: chrome that does not rebuild

A bottom navigation bar is not a screen, it is a frame around screens. ShellRoute gives you a nested Navigator: the Scaffold and its bar are built once, and only the body swaps.

StatefulShellRoute.indexedStack goes further:

each branch keeps its own navigation stack and its own scroll position. Go three screens deep in Chats, switch to Map and come back, and you are still three screens deep, at the same scroll position.

That is the behavior users expect from a tabbed app, and re-implementing it by hand is a common source of navigation bugs.



one Scaffold, three independent stacks

Never nest a Navigator by hand. The shell already is one.

redirect: one guard for the whole app

```
final router = GoRouter(  
  // Re-runs redirect on every sign-in and sign-out.  
  refreshListenable: GoRouterRefreshStream(  
    FirebaseAuth.instance.authStateChanges(),  
  ),  
  redirect: (context, state) {  
    final signedIn = FirebaseAuth.instance.currentUser != null;  
    final atLogin = state.matchedLocation == '/login';  
  
    // Keep where they were headed, so login can finish the trip.  
    if (!signedIn && !atLogin) return '/login?from=${state.uri}';  
    if (signedIn && atLogin) return '/';  
    return null; // null means: stay exactly where you are  
  },  
  routes: [ /* ... */ ],  
);
```

The bridge back to Lecture 7

redirect runs before every navigation, including the very first one and every deep link. One place decides who may see what, so there are no per-screen checks to forget.

refreshListenable is what makes it reactive: without it the guard is only consulted when the user navigates, so a sign-out in the background leaves the private screen on display.

Return null to allow. Return a location to send them elsewhere. Never return the location you are already on: that is an infinite redirect, and go_router reports it.

One tap through all three parts



1 · Permission

The user taps “Nearby”. You show your own rationale sheet, then request location, and handle limited, denied and permanentlyDenied before you draw a map



2 · Route

They tap a friend. `context.go('/chat/alice')` builds the screen. `redirect` has already checked they are signed in, and the same URL works from a push notification



3 · Socket

`ChatScreen` opens `wss://.../chat` with a token, listens, sends, heartbeats, reconnects with jitter, and closes cleanly in `dispose()`

Three subsystems, one user gesture. Each of them fails in a way the user can see, and each of them has exactly one correct place to be handled.

Before next week



Recap

WebSocket for two-way, SSE for one-way, MQTT for devices

connect → ready → listen → send → close, and close in dispose()

wss:// only; heartbeats, jittered backoff, and push for the background

Declare in manifest and plist; handle all six permission states

go_router: routes as data, ids in URLs, one redirect guard



This week

Add a live chat screen to your project over wss://

Make it survive airplane-mode on and off, and a locked screen

Request one real permission with a rationale and a settings path

Replace every Navigator.push with a go_router route



Read more

pub.dev/packages/web_socket_channel

pub.dev/packages/permission_handler

docs.flutter.dev/ui/navigation

RFC 6455 · the WebSocket Protocol

developer.android.com/about/versions/13/behavior-changes-13